

SAML SSO & Automated User Provisioning in Okta

A hands-on identity federation project — documented by Rithik Kumar

For this practice project I set up real SAML-based Single Sign-On between Okta (as Identity Provider) and a target application, plus automated, API-driven user provisioning on top of it — not just login, but create/update/deactivate lifecycle actions triggered from Okta.

I used Salesforce as the target application for this specific demo, since its free Developer Edition is one of the few platforms that lets you configure a genuinely complete SAML + REST provisioning integration without a paid tier. The configuration pattern here — IdP metadata exchange, Connected App / OAuth provisioning setup, attribute mapping, import reconciliation — is the same pattern used for any SAML 2.0 or SCIM-compatible application (Workday, ServiceNow, Slack Enterprise, Zoom, etc.), so this demonstrates the underlying skill, not a Salesforce-specific one.

I also independently tested the OAuth 2.0 / OIDC authorization code flow for a separate Okta app using Postman, confirming token issuance end-to-end — kept out of this write-up to avoid repeating the same login concepts twice.

Official references I followed throughout: Okta's SAML configuration guide for Salesforce, and Okta's REST-based provisioning setup guide (both linked in the GitHub repo README).

PHASE 1 — SAML TRUST CONFIGURATION

Step 1 — Following Okta's Official SAML Setup Guide

Started with Okta's own SAML configuration docs for the Salesforce integration — it lays out exactly what Salesforce needs: the Issuer, the IdP certificate, and the IdP Login URL. I kept this open in one tab the whole time as a reference.

The screenshot shows a web browser window displaying the Okta documentation page for SAML configuration. The address bar shows the URL: `saml-doc.okta.com/SAML_Docs/How-to-Configure-SAML-2.0-in-Salesforce.html?baseAdminUrl=https://integrator-8064210-admin.okta.com&app=salesforce&instanceId=00a16lui...`. The page content includes the following sections:

- SAML Version:** Make sure this is set to 2.0. This should be enabled by default.
- Issuer:** Copy and paste the following:
`exkl6luiq7KJzDdu698`
- Identity Provider Certificate:** Download, then upload the following certificate into this field:
`https://integrator-8064210-admin.okta.com/admin/org/security/00a16luiq7KJzDdu698/cert`
- Identity Provider Login URL:** Copy and paste the following:
`https://integrator-8064210.okta.com/app/salesforce/exkl6luiq7KJzDdu698/sso/saml`

This URL will authenticate your users when they attempt to log in directly to Salesforce or click on a deep link in Salesforce and are not currently authenticated. This is required if you want to enable SP-Initiated SAML authentication.
- Custom Logout URL:** Optional. Copy and paste the following:
`https://integrator-8064210.okta.com`
- API Name:** Enter an API name of your choice.
- Entity ID:**
 - If you have a custom domain setup, use `https://[customDomain].my.salesforce.com`
Note: If you have configured a sandbox environment, don't include `.sandbox` in the custom domain field.
 - If you do not have a custom domain setup, use `https://saml.salesforce.com`
- Click **Save**.

The bottom of the screenshot shows a Windows taskbar with various application icons, a search bar, and system tray icons including the date and time (23:21, 19-08-2026).

Step 2 — SAML SSO Settings Saved on the Salesforce Side

Pasted Okta's Issuer, uploaded the certificate, and set the Identity Provider Login URL inside Salesforce's Single Sign-On Settings. Once saved, Salesforce shows its own Entity ID and the SP endpoints (Login URL, Logout URL, OAuth Token Endpoint) it generated in return — this is the two-way trust handshake between Okta (IdP) and Salesforce (SP) actually completing.

The screenshot displays the Salesforce Setup interface, specifically the 'Single Sign-On Settings' page for SAML Single Sign-On. The left sidebar shows the navigation menu with 'Setup' and 'Object Manager' tabs. The main content area is titled 'SAML Single Sign-On Settings' and includes a 'Back to Single Sign-On Settings' link. The settings are organized into sections: 'SAML Single Sign-On Settings', 'Just-in-time User Provisioning', and 'Endpoints'. The 'SAML Single Sign-On Settings' section contains fields for Name, SAML Version, Issuer, Identity Provider Certificate, Request Signing Certificate, Assertion Decryption Certificate, SAML Identity Type, SAML Identity Location, Service Provider Initiated Request Binding, Identity Provider Login URL, Custom Logout URL, Custom Error URL, Use Salesforce MFA for this SSO Provider, and Single Logout Enabled. The 'Just-in-time User Provisioning' section shows 'User Provisioning Enabled' as checked and 'User Provisioning Type' as 'Standard'. The 'Endpoints' section lists 'Login URL', 'Logout URL', and 'OAuth 2.0 Token Endpoint'. The bottom of the screen shows the Windows taskbar with various application icons and the system clock.

Field	Value
Name	Okta SSO
SAML Version	2.0
Issuer	okta-10a4c76c2da686
Identity Provider Certificate	EMAILADDRESS=info@okta.com, CN=integrator-8064210, OU=SSOProvider, O=Okta, L=San Francisco, ST=California, C=US
Request Signing Certificate	SetSignedCert_19Aug2026_173311
Assertion Decryption Certificate	Assertion not encrypted
SAML Identity Type	Federation ID
SAML Identity Location	Subject
Service Provider Initiated Request Binding	HTTP-POST
Identity Provider Login URL	https://integrator-8064210.okta.com/app/salesforce/okta-10a4c76c2da686/sso/saml
Custom Logout URL	https://integrator-8064210.okta.com
Custom Error URL	
Use Salesforce MFA for this SSO Provider	☐
Single Logout Enabled	☐

Field	Value
User Provisioning Enabled	☒
User Provisioning Type	Standard

Field	Value
Login URL	https://orgfarm-ef6425a25a-dev-ed.develop.my.salesforce.com
Logout URL	https://orgfarm-ef6425a25a-dev-ed.develop.my.salesforce.com/services/auth/sp/saml/logout
OAuth 2.0 Token Endpoint	https://orgfarm-ef6425a25a-dev-ed.develop.my.salesforce.com/services/oauth2/token

PHASE 2 — SETTING UP AUTOMATED PROVISIONING

Step 3 — Reviewing the Target Org's User Model First

Before turning on automated provisioning, I checked my own admin user record in Salesforce to understand the fields Salesforce actually tracks (Profile, Role, License type) — this matters because provisioning has to map correctly against these.

The screenshot shows the Salesforce Setup interface for a user named Rithik Kumar. The left sidebar contains navigation links for various setup areas, including Service Cloud Reports, Commerce Setup Assistant, Hyperforce Assistant, Release Updates, Salesforce Mobile App, Lightning Usage, AgentExchange Offers, Sales Cloud Everywhere, ADMINISTRATION, Users, Analytics Groups, Permission Set Groups, Permission Sets, Profiles, Public Groups, Queues, Roles, User Management Settings, Data, Email, PLATFORM TOOLS, Subscription Management, Apps, Feature Settings, Slack, Data Cloud, Heroku, MuleSoft, and Einstein.

The main content area displays the user's details and settings. The user's name is Rithik Kumar, and their email is rithik.kumar@agentforce.com. The user's role is Salesforce Administrator, and their profile is Salesforce Administrator. The user's license is Salesforce Administrator.

The user's details are as follows:

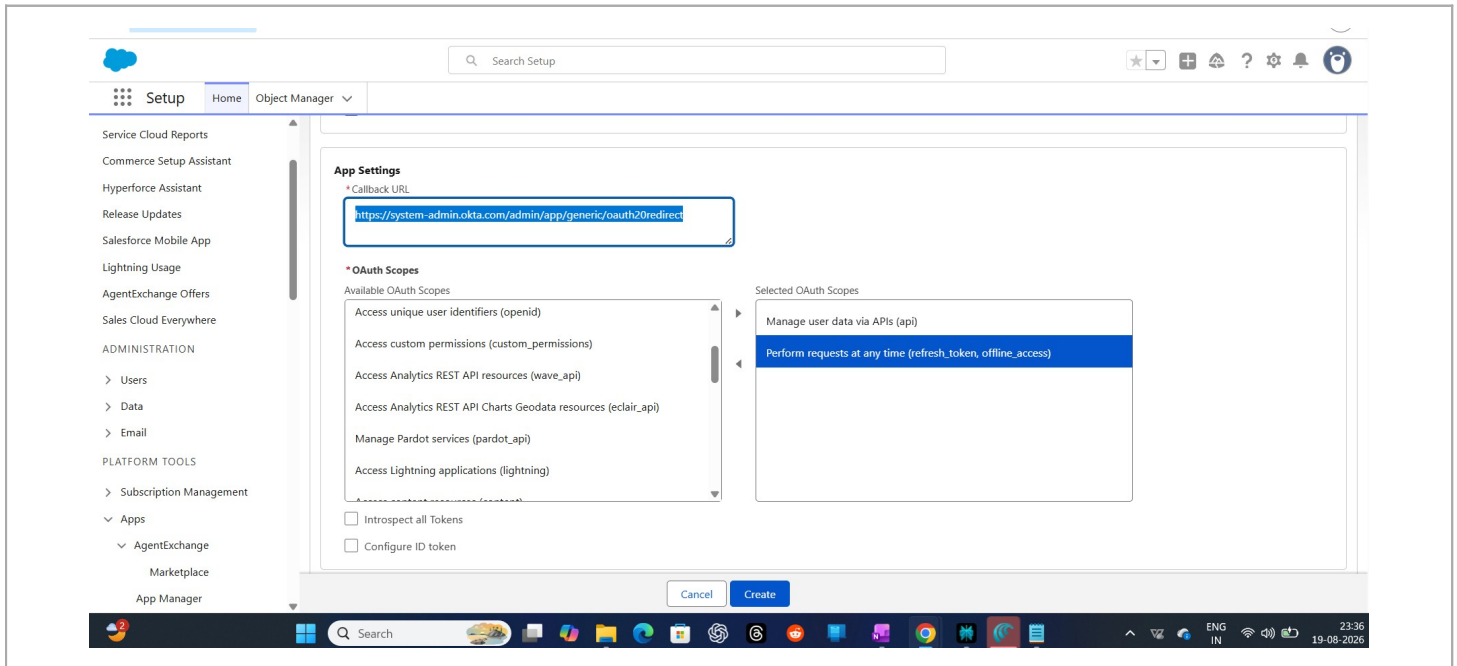
Field	Value
Name	Rithik Kumar
Alias	dk
Email	rithik.kumar@agentforce.com (Verified)
Username	dkumar87753@salesforce.com
Nickname	User17871558377490050001
Title	
Company	Cognizant Technology Solutions
Department	
Division	
Address	
Time Zone	(GMT-07:00) Pacific Daylight Time (America/Los_Angeles)
Locale	English (United States)
Language	English
Delegated Approver	
Manager	
Receive Approval Request Emails	Only if I am an approver
Federation ID	
App Registration: One-Time Password Authenticator	Connected
App Registration: Salesforce Authenticator	Connected
Security Key (U2F or WebAuthn)	Disables
Lightning Login	Disables
Temporary Verification Code (Expires in 1 to 24 Hours)	Disables

The user's settings are as follows:

Setting	Value
Role	Salesforce
User License	Salesforce Administrator
Profile	Salesforce Administrator
Active	☒
Marketing User	☐
Offline User	☐
Knowledge User	☒
Flow User	☐
Service Cloud User	☐
Site.com Contributor User	☐
Site.com Publisher User	☐
WDC User	☐
Mobile Push Registrations	None
Data.com User Type	
Accessibility Mode (Classic Only)	☐
Debug Mode	☐
High-Contrast Palette on Charts	☐
Load Lightning Pages While Scrolling	☒
Send Apex Warning Emails	☐
Salesforce CRM Content User	☒
Receive Salesforce CRM Content Email Alerts	☒
Receive Salesforce CRM Content Alerts as Daily Digest	☒
Make Setup My Default Landing Page	☐
Quick Access Menu	☐
Development Mode	☐
Show View State in Development Mode	☐
Cache Diagnostics	☐
Allow Forecasting	☐
No HTTP Cookies	☐

Step 4 — Creating a Connected App for API-Based Provisioning

SAML handles login, but provisioning (auto-creating/updating/deactivating users) needs a separate OAuth-based connection. Created an External Client App in Salesforce with a callback URL and the OAuth scopes Okta needs to manage users via API.



Step 5 — Cross-Checking Against Okta's REST Provisioning Docs

Referenced Okta's help docs for REST-based Salesforce provisioning to confirm the exact callback URL format and scopes expected — didn't want to guess this part.

The screenshot displays the Okta Docs website interface. The top navigation bar includes the Okta logo, 'Docs', and links for 'Documentation', 'Release Notes', 'Okta Developer', 'Auth0', 'Training', and 'Support'. A search icon and a settings icon are also present. The left sidebar lists various documentation categories, with 'App integrations' expanded to show 'Get started with app integrations', 'Learn about app integrations', 'Add app integrations', and 'Integration guides'. The main content area is titled 'Migrate Connected App to External Client App' and is divided into two tabs: 'Identity Engine' (selected) and 'Classic Engine'. The content is structured as follows:

- 1. Create a new external client app or migrate an existing connected app. [Migrate Connected App to External Client App](#)
- 2. Configure the external client app OAuth settings.
- 3. Configure the external client app OAuth settings.
 - In the **App settings** section, use the following values:
 - Enable OAuth: Enabled
 - Callback URL: <https://system-admin.okta.com/admin/app/generic/oauth2redirect>
 - OAuth Scopes: Manage user data via APIs (api) and Perform requests at any time (refresh_token, offline_access)
 - In the **Security** section, enable the following options:
 - Require secret for Web Server Flow
 - Require secret for Refresh Token Flow
 - Require Proof Key for Code Exchange (PKCE) extension for Supported Authorization Flows
 - Enable Refresh Token Rotation
- 4. In the **Policies** tab, use the following values:
 - Permitted Users:** All users can self-authorize
 - Refresh Token Policy:** Refresh token is valid until revoked

At the bottom of the page, there is a 'Feedback' button and a 'TOP' button. The footer of the browser window shows the taskbar with various application icons and the system clock indicating 23:37 on 19-08-2026.

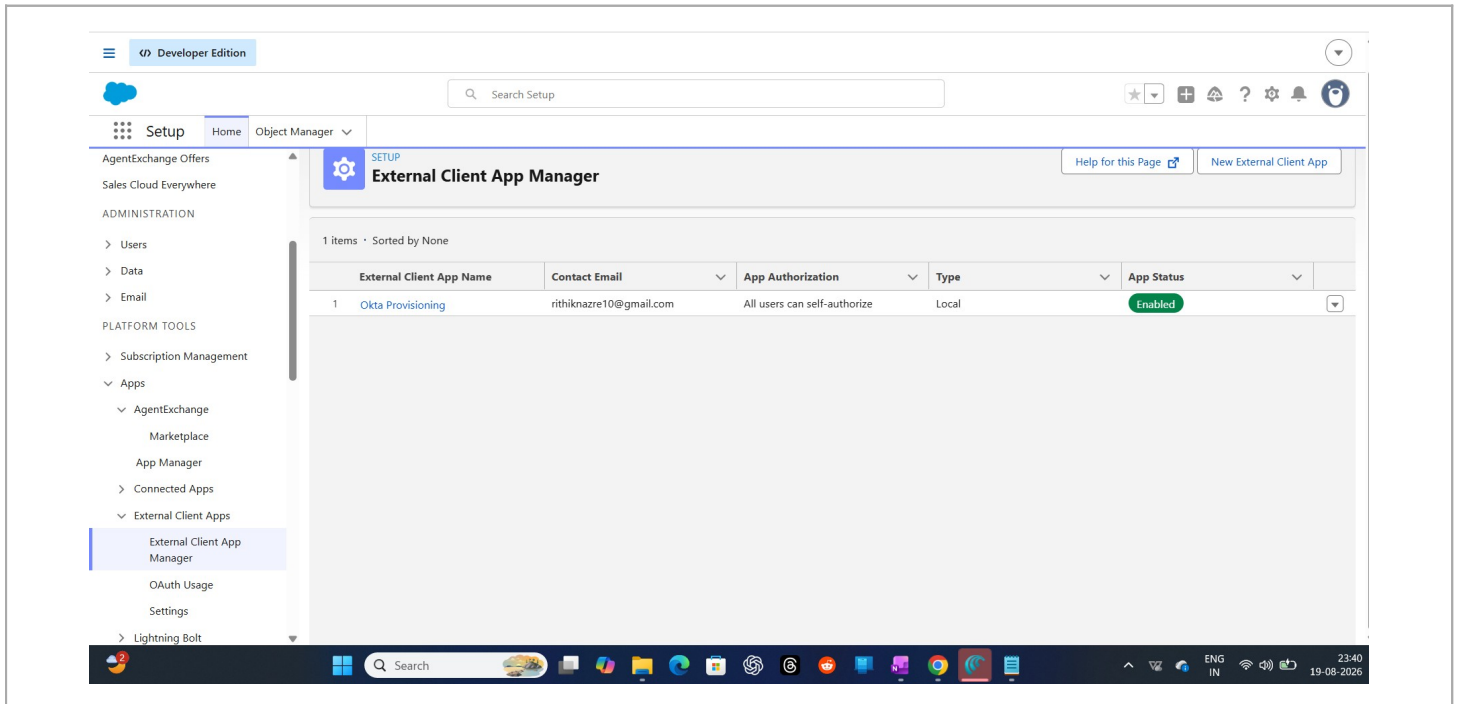
Step 6 — Confirming the Connected App Is Active

Checked the app's policy settings and confirmed its status shows Enabled before trying to connect it from Okta's side.

The screenshot displays the Salesforce Setup interface for the 'Okta Provisioning' app. The left sidebar contains navigation links such as 'Setup', 'Home', 'Object Manager', 'Service Cloud Reports', 'Commerce Setup Assistant', 'Hyperforce Assistant', 'Release Updates', 'Salesforce Mobile App', 'Lightning Usage', 'AgentExchange Offers', 'Sales Cloud Everywhere', 'ADMINISTRATION' (with sub-links for Users, Data, and Email), and 'PLATFORM TOOLS' (with sub-links for Subscription Management, Apps, AgentExchange, and Marketplace). The main content area is titled 'Manage External Client Apps' and 'Okta Provisioning'. It shows the 'Contact Email' as 'rithiknazre10@gmail.com', 'App Authorization' as 'All users can self-authorize', and 'App Status' as 'Enabled'. Below this, the 'Policies' tab is selected, showing a configuration area for 'App Policies' with a 'Start Page' dropdown set to 'None'. The bottom of the screen shows a Windows taskbar with various application icons and a system clock indicating 22:38 on 19-08-2026.

Step 7 — Verifying It Shows Up in Salesforce's App Manager

Double-checked in External Client App Manager that the app is listed and enabled — small sanity check before switching over to Okta.

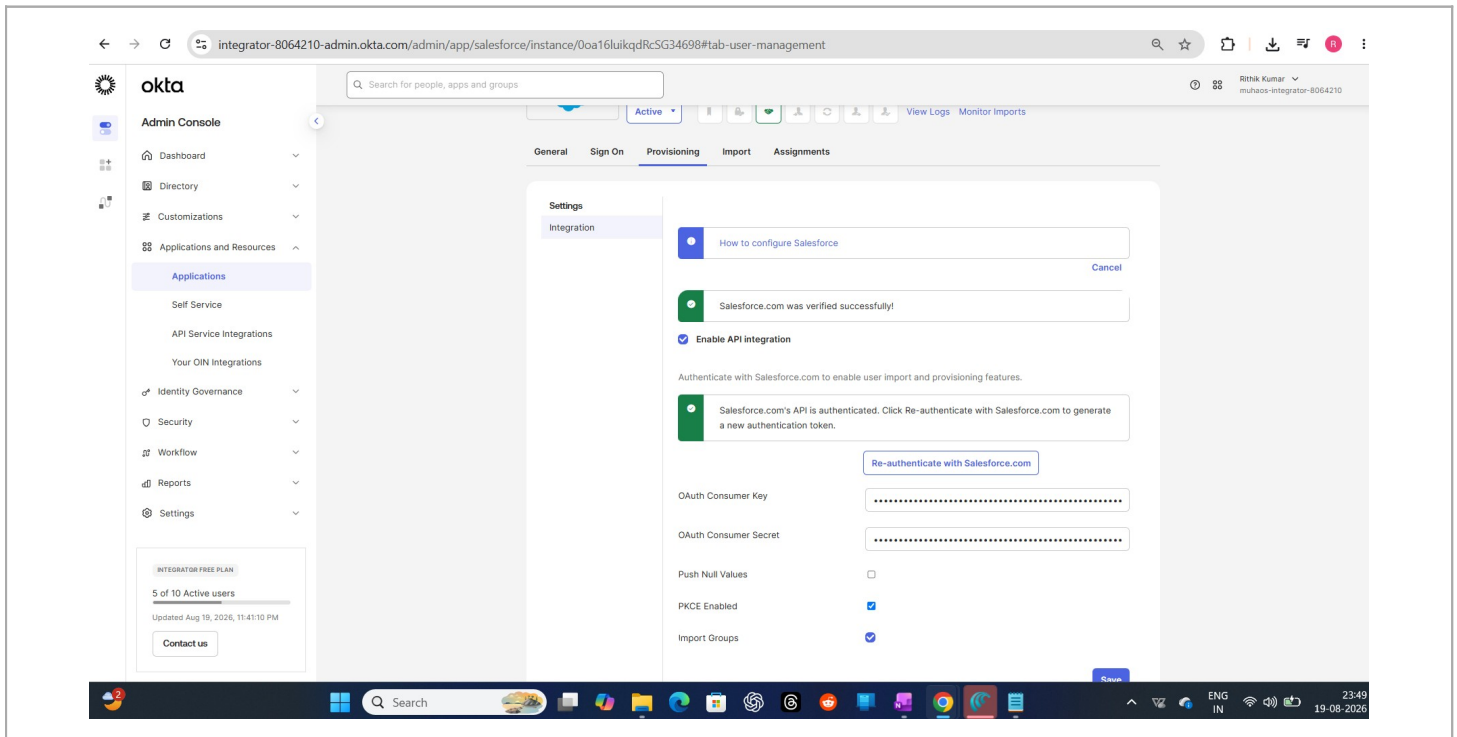


The screenshot shows the Salesforce Developer Edition Setup page for the External Client App Manager. The left sidebar contains a navigation menu with categories like AgentExchange Offers, Sales Cloud Everywhere, ADMINISTRATION, and PLATFORM TOOLS. The 'External Client App Manager' option is selected under the 'Apps' category. The main content area displays a table with one item, 'Okta Provisioning', which is enabled. The table columns are External Client App Name, Contact Email, App Authorization, Type, and App Status. The 'Okta Provisioning' app is listed with contact email 'rithiknazre10@gmail.com' and authorization 'All users can self-authorize'. The app status is 'Enabled'.

External Client App Name	Contact Email	App Authorization	Type	App Status
1 Okta Provisioning	rithiknazre10@gmail.com	All users can self-authorize	Local	Enabled

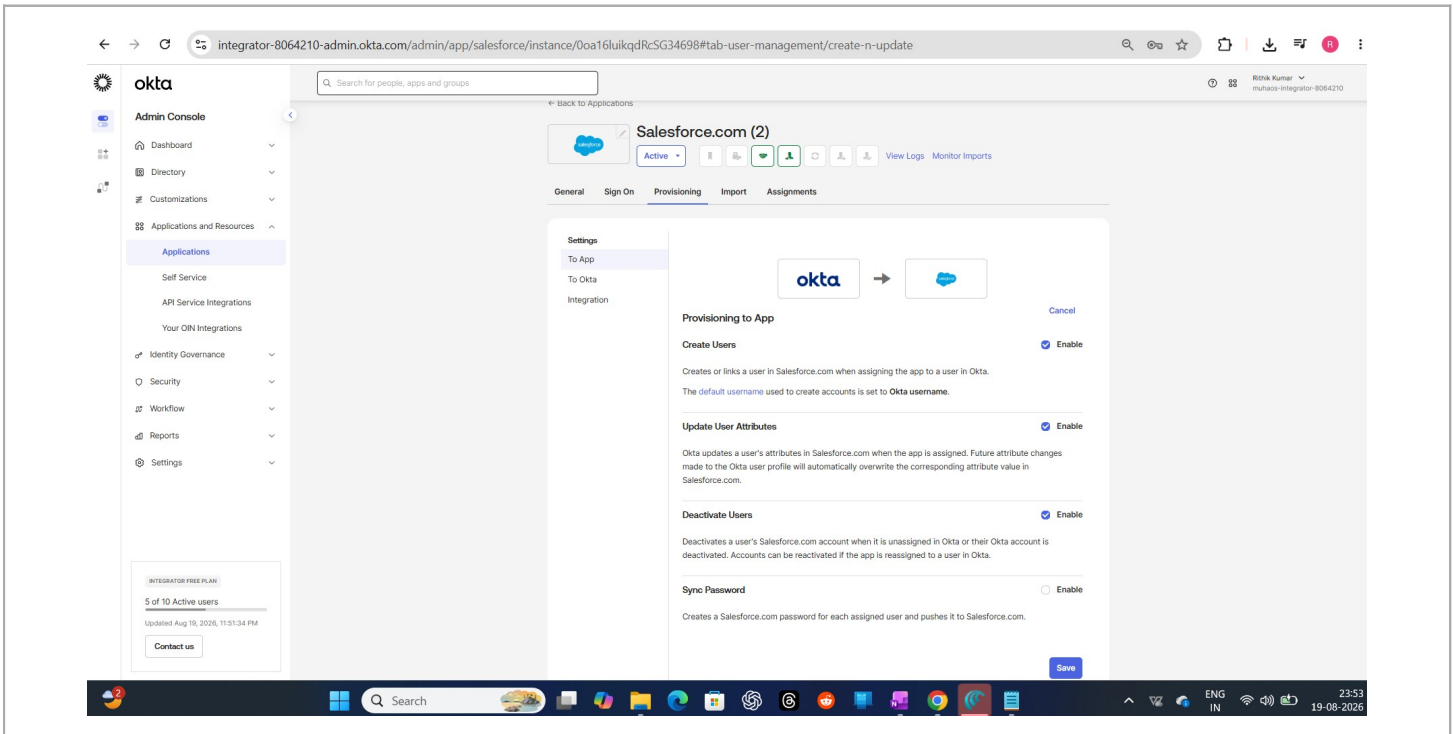
Step 8 — Authenticating Okta Against Salesforce's API

Back in Okta, enabled API integration on the Salesforce app and authenticated using the Connected App's OAuth Consumer Key and Secret. (Not sharing the actual key/secret values here — blurred out in the screenshot, and I'll be deleting this dev app after finishing this practice project anyway.) PKCE and Import Groups are both enabled.



Step 9 — Turning On Create / Update / Deactivate

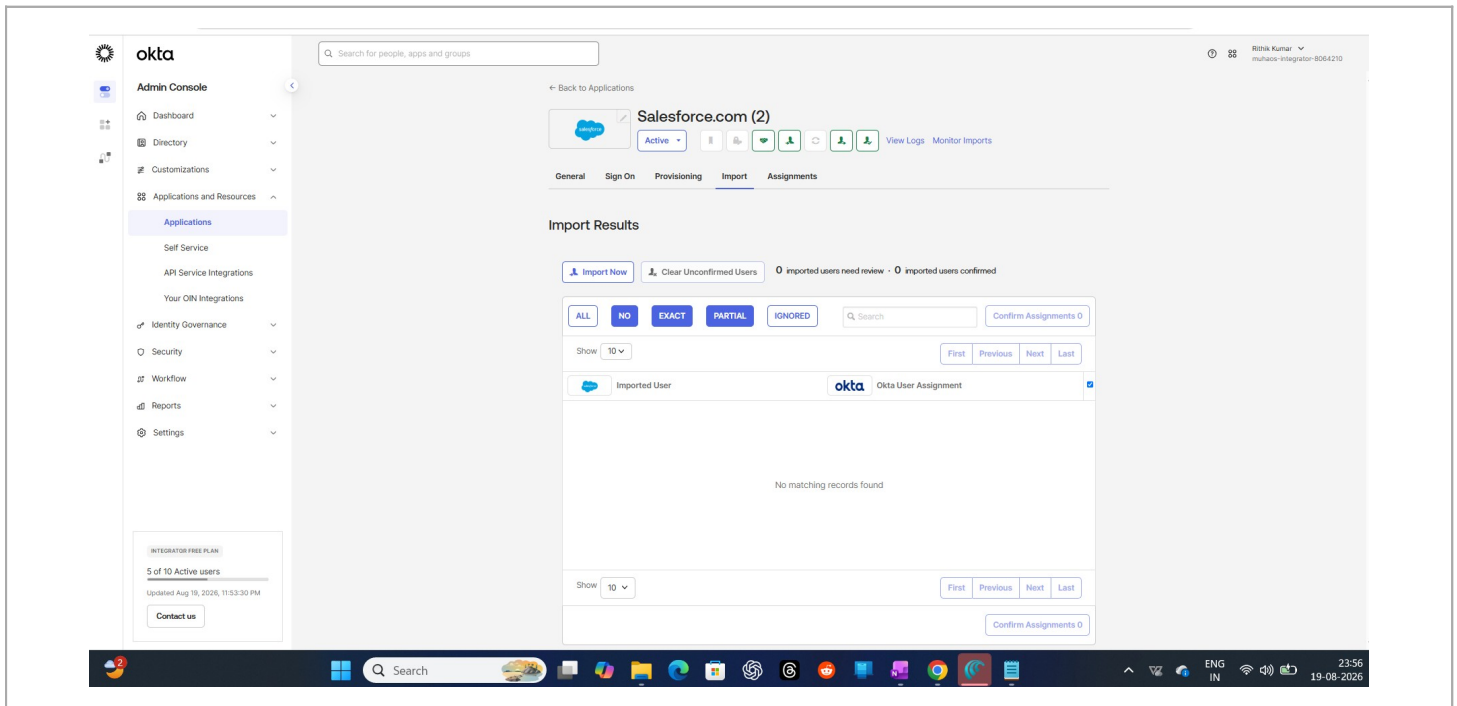
Enabled the three core provisioning actions on the Okta side: Create Users, Update User Attributes, and Deactivate Users — this is what actually makes JML-style automation possible against Salesforce, not just SSO login.



PHASE 3 — IMPORT & RECONCILIATION

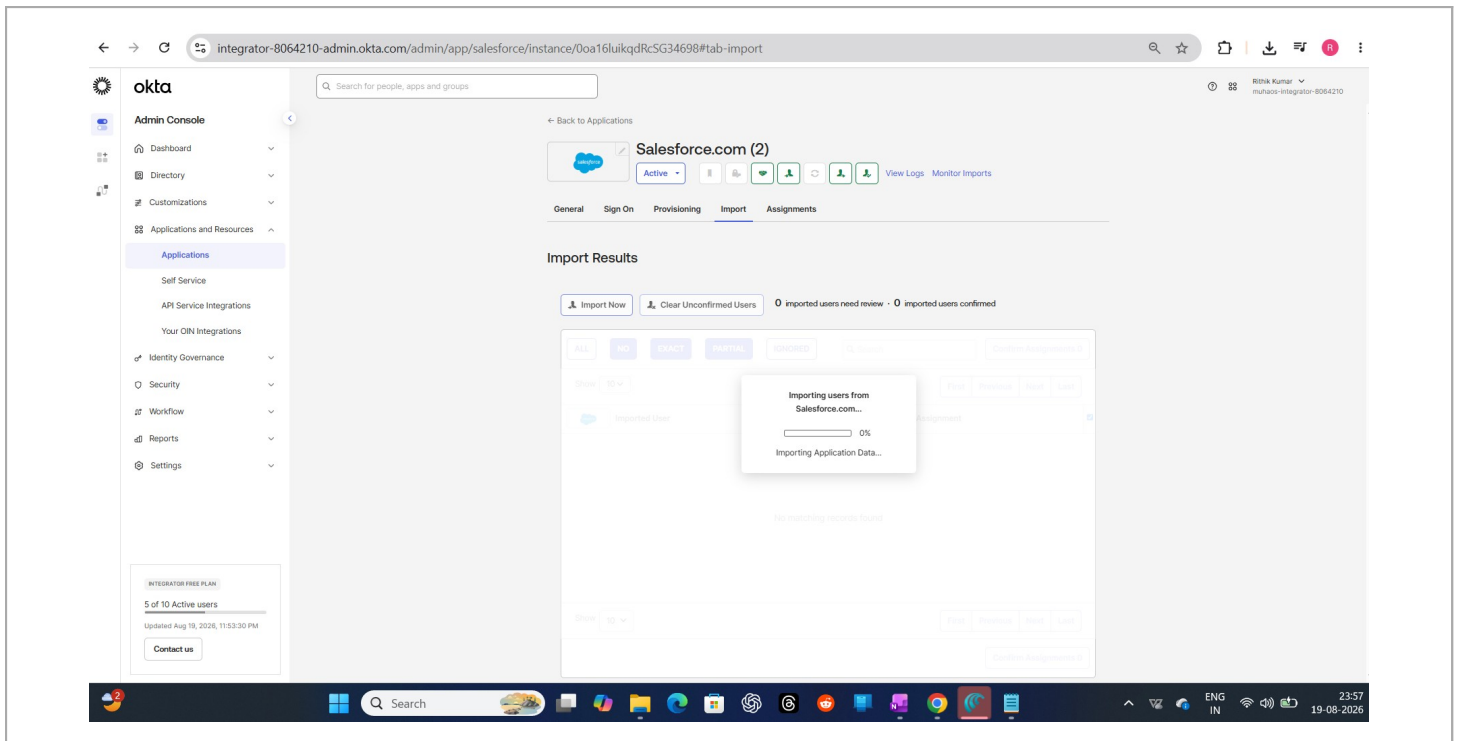
Step 10 — Running the First Import (Nothing Yet)

Ran an initial import from Salesforce into Okta to see what user data would come across. At this point, nothing had synced yet — clean starting point.



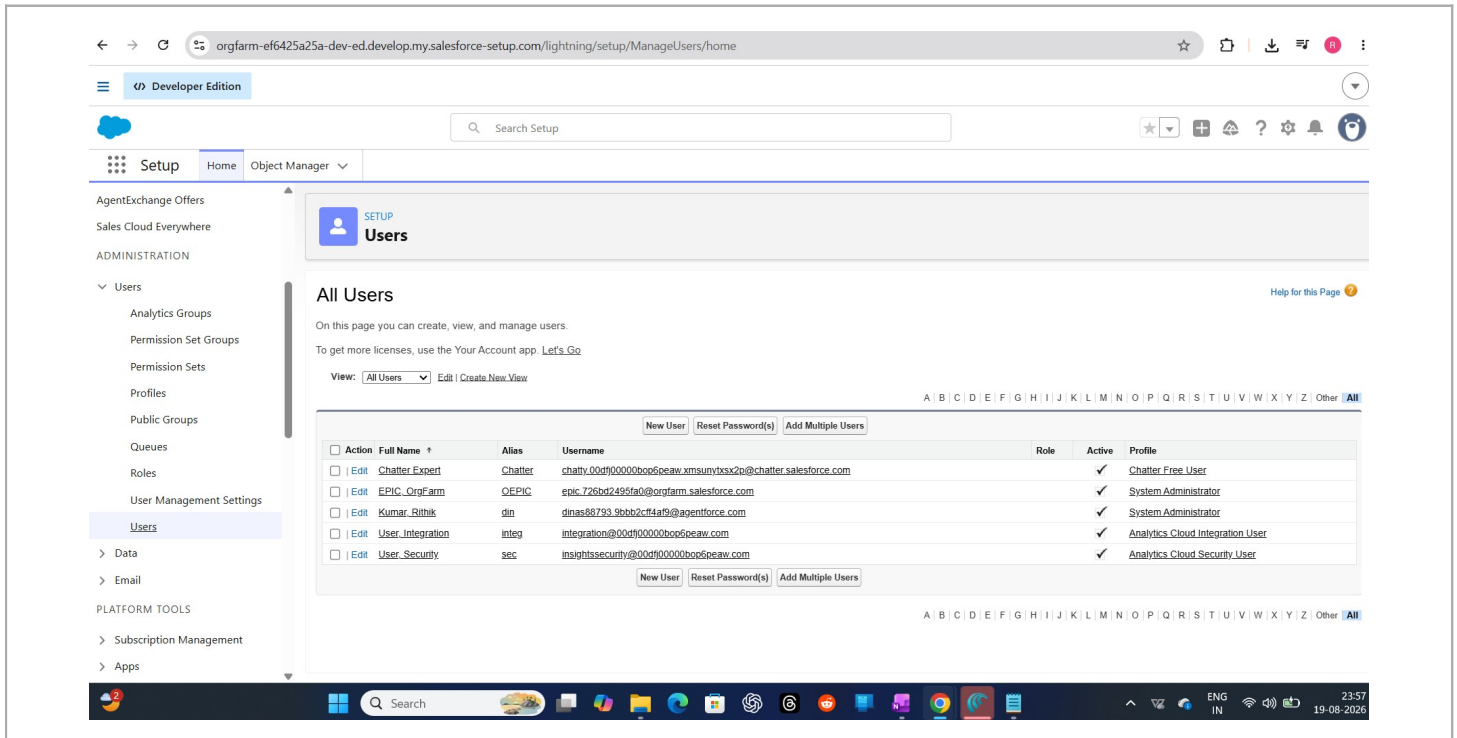
Step 11 — Import In Progress

Kicked off the real import — Okta pulling account data live from Salesforce.



Step 12 — What Salesforce Actually Has on Its Side

Checked Salesforce's own user list directly to see what existed there already (a mix of default system users like Chatter Expert, an integration user, and my own admin account) — useful to know what the import was about to try matching against.



The screenshot shows the Salesforce Setup interface for a Developer Edition instance. The left sidebar contains navigation links for Setup, Home, Object Manager, and various administrative tools. The main content area is titled 'Users' and displays a list of all users in the system. The list includes columns for Action, Full Name, Alias, Username, Role, Active status, and Profile. Five users are listed: Chatter Expert, EPIC_CrgFarm, Kumar_Rohik, User_Integration, and User_Security. Each user has an 'Edit' link and a checkbox for selection. The 'User_Integration' and 'User_Security' users are marked as active and have specific profiles assigned.

Action	Full Name	Alias	Username	Role	Active	Profile
☐ Edit	Chatter Expert	Chatter	chatty.00df00000bop6peaw.xmsunvdxs2p@chatter.salesforce.com		✓	Chatter Free User
☐ Edit	EPIC_CrgFarm	QEPIC	epic.726bd24956a0@orgfarm.salesforce.com		✓	System Administrator
☐ Edit	Kumar_Rohik	din	dinas88793.9bbb2c4f4a9@agentforce.com		✓	System Administrator
☐ Edit	User_Integration	integ	integration@00df00000bop6peaw.com		✓	Analytics Cloud Integration User
☐ Edit	User_Security	sec	insightssecurity@00df00000bop6peaw.com		✓	Analytics Cloud Security User

Step 13 — Reviewing the Match Results

Okta's import screen shows exactly how each Salesforce account was evaluated: most came back as "NO Okta user matches found" (meaning they'd become new Okta identities), but my own account came back as a **PARTIAL** match — useful for seeing how reconciliation logic actually behaves with real, messy data instead of a clean demo dataset.

The screenshot displays the Okta Admin Console interface for the 'integrator-8064210-admin.okta.com' instance. The left sidebar shows the 'Admin Console' menu with options like Dashboard, Directory, Customizations, Applications and Resources, Self Service, API Service Integrations, Your OIN Integrations, Identity Governance, Security, Workflow, Reports, and Settings. The main content area is titled 'Imported Users' and shows a table of match results. The table has columns for 'Imported User' and 'Okta User Assignment'. The 'Imported User' column lists users with their Name, Username, and Email. The 'Okta User Assignment' column shows the matching Okta user or a conflict message. The results are as follows:

Imported User	Okta User Assignment
NO Okta user matches found Name: Chatter Expert Username: chatter.00df00000bop6peaw.xms... Email: noreply@example.com	NEW Okta user Name: Chatter Expert Username: noreply@example.com Email: noreply@example.com This choice creates a conflict
NO Okta user matches found Name: Orgfarm EPIC Username: epic.728b2495fa0@orgfarm.sale... Email: epic.orgfarm@salesforce.com	NEW Okta user Name: Orgfarm EPIC Username: epic.orgfarm@salesforce.com Email: epic.orgfarm@salesforce.com
1 PARTIAL Okta user match found Name: Ritvik Kumar Username: dines80793.9ebb2c7f4a9@gagentf... Email: dines80793@hutdot.com	PARTIAL Okta user match Name: Ritvik Kumar Username: nobere6404@muhaos.com Email: nobere6404@muhaos.com
NO Okta user matches found Name: Integration User Username: integration@00df00000bop6pea... Email: noreply@example.com	NEW Okta user Name: Integration User Username: noreply@example.com Email: noreply@example.com This choice creates a conflict
NO Okta user matches found Name: Security User Username: insightssecurity@00df00000bop... Email: noreply@example.com	NEW Okta user Name: Security User Username: noreply@example.com Email: noreply@example.com This choice creates a conflict

The bottom of the screen shows a Windows taskbar with various application icons and a system clock indicating 23:58 on 19-08-2026.

PHASE 4 — ASSIGNMENT & PROVISIONING IN ACTION

Step 14 — Assigning the App to Trigger Provisioning

Assigned the Salesforce app to my own Okta user. This is the action that actually fires the Create User provisioning call over to Salesforce.

The screenshot displays the Okta Admin Console interface for the 'Salesforce.com (2)' application. The left sidebar shows the 'Admin Console' menu with options like Dashboard, Directory, Customizations, Applications and Resources, Self Service, API Service Integrations, Your OIN Integrations, Identity Governance, Security, Workflow, and Reports. The main content area is titled 'Salesforce.com (2)' and shows the 'Assign' button, a search bar, and a table of assigned users. The table lists one user, 'Rithik Kumar', with the email 'nobre6404@muhaos.com' and a type of 'Individual'. The right sidebar contains 'REPORTS' (Current Assignments, Recent Unassignments) and 'SELF SERVICE' (You need to enable self service for org managed apps before you can use self service for this app. Go to self service settings). The bottom status bar shows '© 2026 Okta, Inc.', 'Privacy', 'Status site', 'OK14 US Cell', 'Version 2026.08.1 E', 'Feedback', and the date '20-08-2026'.

Okta Admin Console interface showing the 'Salesforce.com (2)' application configuration. The 'Assign' button is visible, indicating the process of assigning the app to users. The interface includes a search bar, filters, and a table of assigned users (Person, Type, Name, Email).

Assigned Users Table:

Person	Type	Name	Email
Individual	Individual	Rithik Kumar	nobre6404@muhaos.com

Additional details visible in the interface include the 'Assign' button, 'Convert assignments' dropdown, and the 'SELF SERVICE' section indicating that self-service is disabled for this app.

Step 15 — Confirming the User Was Really Created in Salesforce

Went back into Salesforce's own user list and found the new record Okta had just provisioned automatically (highlighted) — correct email, correct username, Standard User profile applied. This is the actual proof that the automation works end-to-end, not just configured on paper.

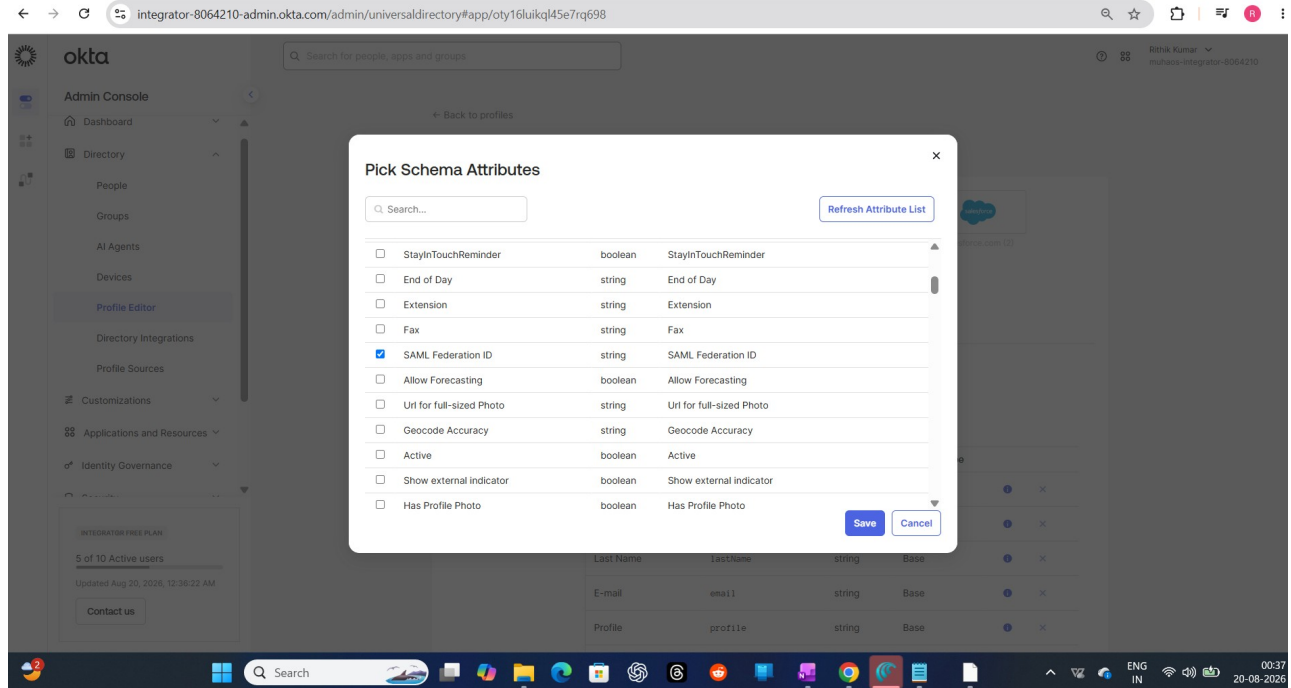
The screenshot shows the Salesforce Setup interface. The left sidebar contains navigation links for Setup, Home, Object Manager, and various administrative tools. The main content area is titled 'Users' and displays a table of all users. The user 'Kumar, Rithik' is highlighted in yellow, indicating the newly provisioned user. The table columns include Action, Full Name, Alias, Username, Role, Active, and Profile.

Action	Full Name	Alias	Username	Role	Active	Profile
☐ Edit	Chatter Expert	Chatter	chatty00df00000bop6peaw.xmsunytbx2p@chatter.salesforce.com		✓	Chatter Free User
☐ Edit	EPIC_OrgFarm	OEPIE	epic.726bd2495fa0@orgfarm.salesforce.com		✓	System Administrator
☐ Edit	Kumar, Rithik	din	dinas88793.9bbb2c7f4a9@agentforce.com		✓	System Administrator
☐ Edit	Kumar, Rithik	nobere64	nobere6404@muhaos.com		✓	Standard User
☐ Edit	User_Integration	integ	integration@00df00000bop6peaw.com		✓	Analytics Cloud Integration User
☐ Edit	User_Security	sec	insightssecurity@00df00000bop6peaw.com		✓	Analytics Cloud Security User

PHASE 5 — ATTRIBUTE MAPPING

Step 16 — Mapping Profile Attributes Between Systems

Opened Okta's Profile Editor for the Salesforce app and picked which schema attributes should flow between the two systems, including SAML Federation ID — this is what ties the SSO login identity to the correct provisioned account.



Step 17 — Verifying firstName / lastName / email Actually Map Correctly

Checked the live mapping between Salesforce's appuser fields and Okta's user profile — confirmed my real first name, last name, and email were correctly flowing across, not placeholder values.

The screenshot displays the Okta Admin Console interface. On the left, the 'Admin Console' sidebar is visible with options like Dashboard, Directory, People, Groups, AI Agents, Devices, Profile Editor, Directory Integrations, Profile Sources, Customizations, Applications and Resources, and Identity Governance. The main content area shows the 'Salesforce.com (2) User Profile' and 'Okta User Profile' side-by-side. The Salesforce profile lists fields like 'appuser.firstName', 'appuser.lastName', and 'appuser.email'. The Okta profile lists corresponding fields like 'login', 'middleName', 'honorificPrefix', 'honorificSuffix', 'title', 'displayName', 'nickname', 'profileUrl', 'secondEmail', 'mobilePhone', and 'primaryPhone'. The mapping shows that 'appuser.firstName' maps to 'login' with the value 'Rithik', 'appuser.lastName' maps to 'middleName' with the value 'Kumar', and 'appuser.email' maps to 'secondEmail' with the value 'nobere6404@muhaos.com'. The bottom of the screen shows a Windows taskbar with various application icons and a system tray indicating the time as 00:40 on 20-08-2026.

Salesforce.com (2) User Profile	Okta User Profile
Username is set by Salesforce.com (2)	login string
appuser.firstName string	Rithik string
appuser.lastName string	Kumar string
	middleName string
	honorificPrefix string
	honorificSuffix string
appuser.email string	nobere6404@muhaos.com email
	title string
	displayName string
	nickname string
	profileUrl uri
	secondEmail email
	mobilePhone string
	primaryPhone string

Step 18 — Final Provisioned User Record in Salesforce

Opened the provisioned user's record directly in Salesforce to confirm everything landed correctly — profile, license type, active status, all set as expected by the automation.

The screenshot displays the Salesforce 'User Edit' interface. The left sidebar shows the navigation menu with 'Setup' selected, and 'Users' under 'ADMINISTRATION'. The main content area is titled 'User Edit' and contains the 'General Information' tab. The user's details are as follows:

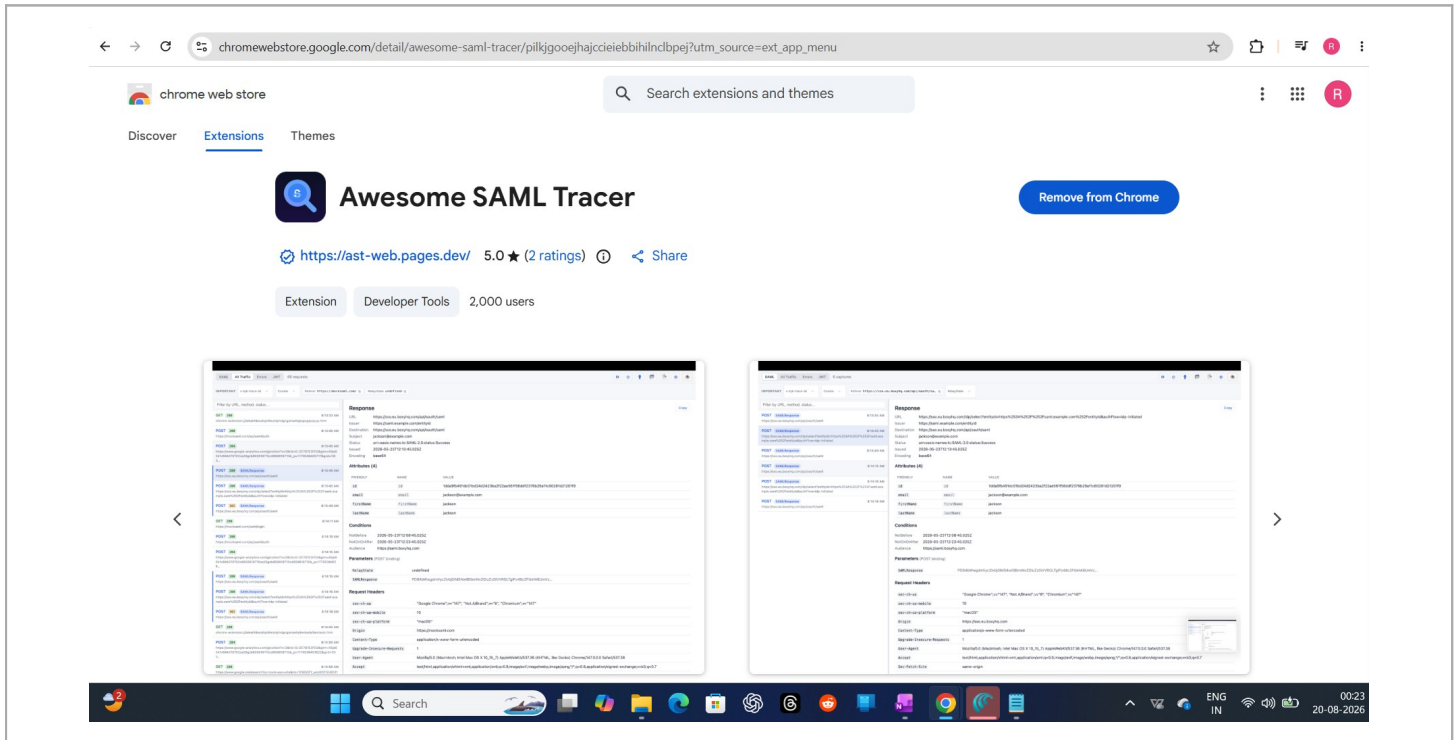
Field	Value
First Name	Ritvik
Last Name	Kumar
Alias	nobere64
Email	nobere6404@muhaos.com
Username	nobere6404@muhaos.com
Nickname	User178716450843935132
Title	
Company	
Department	
Division	
Role	<None Specified>
User License	Salesforce
Profile	Standard User
Active	☒
Marketing User	☐
Offline User	☐
Knowledge User	☐
Flow User	☐
Service Cloud User	☐
Site.com Contributor User	☐
Site.com Publisher User	☐
WDC User	☐
Data.com User Type	--None--
Data.com Monthly Addition Limit	Default Limit (300)
Accessibility Mode (Classic Only)	☐
High-Contrast Palette on Charts	☐

The bottom of the screen shows the Windows taskbar with various application icons and the system clock indicating 00:41 on 20-08-2026.

PHASE 6 — INDEPENDENT VALIDATION

Step 19 — Installing a SAML Tracer for Third-Party Verification

Rather than just trusting that the integration "looked right" in the admin consoles, I installed the Awesome SAML Tracer Chrome extension to independently inspect the actual SAML assertions being exchanged during login — then cross-checked that output against Salesforce's own SAML Assertion Validator tool. This lets you pinpoint exactly which part of the exchange is failing if something ever breaks, instead of re-checking the whole integration blind.



What This Project Actually Demonstrates

This wasn't just flipping a toggle labeled "Enable SSO." It covers the full real-world sequence: establishing IdP/SP trust via SAML metadata exchange, setting up a separate OAuth-based Connected App for provisioning, running an import to reconcile existing target-system accounts against Okta identities (including handling partial matches), triggering live provisioning, verifying attribute mapping end-to-end, and independently validating the SAML assertion with a third-party tool instead of just trusting the admin UI.

The reconciliation step in Phase 3 was the most useful part to actually do hands-on — real orgs never have a clean slate, and seeing how Okta classifies existing accounts as new / partial / exact matches is exactly the kind of judgment call that comes up in real migrations.